

Projeto Final de Análise de Projeto e Sistemas

Documento técnico de apoio e handoff

Tatyanne Guedes • Lavynia Monteiro • Daniel Costa

Projeto acadêmico | 10 de novembro de 2026

Salões de beleza e barbearias independentes

Sumário

Bloco 1 Concepção e escopo

Bloco 2 Requisitos

Bloco 3 Arquitetura de software

Bloco 4 Diagramas UML

Bloco 5 Banco de dados

Bloco 7 Planejamento e estimativas

Bloco 8 Documentação para o desenvolvimento

Bloco 9 Qualidade e processo

Fontes e limitações da estimativa

Anexo visual do banco de dados

Anexo visual do contrato da API

Documento de apoio e handoff técnico. Integrantes: Tatyanne Guedes, Lavynia Monteiro e Daniel Costa. Domínio: agendamento de serviços para salões de beleza e barbearias independentes.

Este documento apresenta um sistema planejado para organizar horários, profissionais, clientes e lista de espera. Ele segue os blocos do trabalho do professor e reúne requisitos, arquitetura, diagramas, modelo de dados, contrato da API, SQL inicial, README e fluxo Git. A aplicação ainda não foi implementada; metas de desempenho e capacidade são critérios propostos, não resultados medidos. A entrega é autossuficiente e não depende de um repositório remoto.

Bloco 1 Concepção e escopo

Visão. Fila Certa, “Seu horário organizado, seu atendimento no tempo certo”, é uma aplicação web para salões de beleza e barbearias independentes. Hoje, marcações chegam por telefone e mensagens, e a equipe anota horários em agendas separadas. Isso causa reservas duplicadas, esquecimentos, períodos ociosos e dificuldade para oferecer um horário que acabou de ser liberado. O sistema centraliza a agenda e, quando um atendimento é cancelado, oferece o horário à próxima pessoa compatível na lista de espera.

Escopo da versão 1. Cadastro de serviços e profissionais; associação entre eles; configuração de horários semanais e exceções; consulta de horários livres; marcação e cancelamento; lista de espera por serviço e período; oferta temporária do horário liberado; aceite ou recusa pelo cliente; agenda da equipe e relatório diário. O cliente precisa de conta para marcar ou entrar na lista. Ficam fora pagamento, prontuário, estoque de produtos, franquias, integração com redes sociais, aplicativo nativo, lembrete por SMS/WhatsApp e otimização automática de agenda. O MVP mostra a oferta dentro da área do cliente; envio por e-mail é uma melhoria posterior.

Premissas de dimensionamento. Piloto com um estabelecimento, até 15 profissionais, 30 serviços e 2.000 clientes cadastrados; até 50 sessões internas simultâneas. O teste inicial de disponibilidade cobre 100 sessões simultâneas. Expansão para múltiplos estabelecimentos ou franquias depende de nova análise de isolamento e capacidade. Os horários são armazenados com fuso explícito; o estabelecimento opera em um fuso configurado.

Persona	Idade e contexto	Motivação
Renata Alves	39, proprietária de salão	Reduzir horários vazios e acompanhar a agenda sem depender de várias conversas.
Paula Nascimento	27, recepcionista	Marcar, remarcar e cancelar sem criar conflito entre profissionais.
Diego Martins	31, barbeiro	Consultar apenas sua agenda e registrar atendimentos concluídos ou faltas.
Camila Rocha	24, cliente	Encontrar horário compatível e receber uma oportunidade quando alguém cancela.
Marcos Lima	34, responsável técnico	Administrar contas, permissões, segurança e suporte do sistema.

As personas são hipóteses para orientar o projeto, não resultado de pesquisa de campo. O papel CUSTOMER usa conta própria para consultar e administrar as próprias marcações. ADMIN, RECEPTIONIST e PROFESSIONAL são papéis internos, com acesso limitado às funções necessárias.

Stakeholder	Interesse	Influência
Proprietária do estabelecimento	ocupação da agenda, custos e qualidade	alta
Recepção	marcações e alterações sem conflito	alta
Profissionais	agenda correta e intervalos respeitados	média
Clientes	encontrar e administrar horários	média
Equipe de desenvolvimento	regras e contrato estáveis	alta
Provedor de e-mail futuro	entrega de notificações	baixa na V1

Viabilidade técnica. Um monólito modular em Java 21/Spring Boot, PostgreSQL 16 e interface web HTML/CSS/JavaScript é suficiente para o piloto. O PostgreSQL protege contra dois agendamentos simultâneos do mesmo profissional no mesmo intervalo. Microserviços e aplicativo nativo são desnecessários antes de haver escala medida. **Viabilidade operacional.** A recepção configura serviços, profissionais e disponibilidade; um piloto de duas semanas compara a agenda digital com o processo atual. **Viabilidade financeira.** O cenário de infraestrutura de referência está no Bloco 7; valores são estimativas e precisam ser cotados antes de contratar.

Risco inicial	Probabilidade	Impacto	Mitigação
Dois clientes tentam reservar o mesmo horário	média	alto	transação, restrição de sobreposição e resposta 409
Disponibilidade semanal é cadastrada incorretamente	média	alto	revisão pela recepção e prévia dos horários antes de publicar
Cliente não vê a oferta da lista de espera a tempo	média	médio	prazo explícito, painel de ofertas e possibilidade de contato manual pela recepção
Cancelamentos frequentes deixam intervalos ociosos	média	médio	lista de espera compatível e relatório de ocupação
Exposição de dados pessoais ou da agenda	baixa	alto	RBAC, sessões seguras, minimização e auditoria

Risco inicial	Probabilidade	Impacto	Mitigação
Indisponibilidade do serviço em horário de pico	baixa	alto	backup, monitoramento e procedimento manual de contingência

Bloco 2 Requisitos

MoSCoW separa o que é essencial (Must), importante mas adiável (Should), opcional (Could) e fora da V1 (Won't). A prioridade reflete a possibilidade de atender, cancelar e preencher horários liberados com segurança.

ID	Requisito funcional	Prioridade e justificativa
RF001	Autenticar usuários por e-mail e senha	Must: protege agenda e dados pessoais
RF002	Administrar usuários e papéis internos	Must: limita acesso por função
RF003	Cadastrar e atualizar clientes	Must: identifica quem marca o horário
RF004	Cadastrar e atualizar serviços, duração e intervalo de preparação	Must: define quanto tempo cada atendimento ocupa
RF005	Cadastrar profissionais e associá-los a serviços	Must: identifica quem pode realizar cada serviço
RF006	Configurar disponibilidade semanal e exceções	Must: define expediente, folgas e bloqueios
RF007	Consultar horários livres por serviço, profissional e período	Must: permite escolher horário válido
RF008	Criar agendamento para cliente autenticado ou recepção	Must: registra o compromisso sem duplicidade
RF009	Consultar agenda por profissional, período e estado	Must: orienta o trabalho diário
RF010	Cancelar agendamento antes do horário de início	Must: libera a agenda e inicia a oferta da vaga
RF011	Incluir cliente em lista de espera por serviço e período	Must: registra quem quer uma vaga compatível
RF012	Oferecer vaga liberada à próxima pessoa elegível	Must: conecta cancelamento e lista de espera
RF013	Aceitar ou recusar oferta dentro do prazo	Must: confirma quem ocupará a vaga
RF014	Registrar atendimento concluído ou falta	Must: encerra o ciclo e mantém histórico
RF015	Consultar relatório diário de ocupação e cancelamentos	Should: pode começar com consulta da agenda
RF016	Permitir que o cliente consulte e atualize seus próprios dados de contato	Should: recepção pode corrigir dados no piloto
RF017	Enviar lembretes por e-mail	Could: painel do cliente cobre a V1
RF018	Receber pagamento ou sinal pelo sistema	Won't: exige integração financeira e política de reembolso
RF019	Exportar agenda em CSV/ICS	Could: consulta na tela atende o piloto
RF020	Exibir painel resumido de ocupação e faltas	Could: relatório diário cobre a V1

ID	Meta mensurável e verificação
RNF001	p95 de consulta de horários <=300 ms com 50 sessões internas e base de 100 mil agendamentos; medir em staging.
RNF002	Testar 100 sessões simultâneas no piloto; expansão exige teste e redimensionamento.
RNF003	Meta de disponibilidade observada de 99,5% ao mês no piloto, sem SLA contratual; instância única não garante a meta. Duas falhas mensais seguidas acionam revisão de arquitetura e custo.
RNF004	100% do tráfego externo em HTTPS com TLS 1.2 ou superior.
RNF005	Após cinco falhas consecutivas, bloquear a conta por 15 minutos; login válido zera contador e limite por IP complementa a proteção.
RNF006	100% das rotas privadas com autorização no servidor e testes por papel.
RNF007	Todo dado pessoal tem finalidade, acesso e retenção documentados antes da produção.
RNF008	Dois agendamentos ativos do mesmo profissional não podem se sobrepor, inclusive sob requisições concorrentes.
RNF009	Backup diário, retenção de 30 dias e restauração testada trimestralmente.
RNF010	RPO alvo <=24 h e RTO alvo <=4 h, medidos em exercício de recuperação.
RNF011	Marcar ou cancelar um horário em até cinco etapas, verificado com usuários do piloto.
RNF012	Suporte às duas últimas versões estáveis de Chrome, Firefox e Edge.
RNF013	Auditoria de alterações de agenda e permissões mantida por 12

ID	Meta mensurável e verificação
	meses, com acesso restrito.
RNF014	Cobertura >=70% das regras de negócio e 100% das transições críticas de agendamento testadas.
RNF015	Fluxo principal utilizável em telas a partir de 360 px, sem rolagem horizontal.

As 15 histórias estimadas somam 61 pontos: 14 Must e US015, de relatório, como Should. RF016 a RF020 permanecem fora do incremento estimado; o contrato e o modelo V1 não prometem essas funções.

História	Como quero para	Dado/Quando/Então	SP
US001	Usuário entrar com segurança para usar seu perfil	Dadas credenciais válidas, quando entrar, então recebe sessão conforme papel	3
US002	Admin gerir contas para controlar acesso	Dado papel permitido, quando criar ou desativar conta, então a alteração é auditada	5
US003	Recepção cadastrar cliente para registrar marcação	Dados válidos produzem cliente consultável e ligado à conta	3
US004	Recepção cadastrar serviço para publicar duração	Duração válida e nome único ficam disponíveis para agendamento	3
US005	Recepção associar profissional e serviço para ofertar opções reais	Associação ativa permite consultar horários compatíveis	3
US006	Recepção configurar expediente e exceções para controlar disponibilidade	Regra semanal e bloqueio são aplicados ao cálculo de horários	5
US007	Cliente consultar horários para escolher uma opção	Resultado respeita duração, intervalo, expediente e bloqueios	5
US008	Cliente marcar horário para reservar atendimento	Um único agendamento ativo ocupa o intervalo do profissional	5
US009	Recepção consultar agenda para organizar o dia	Filtros retornam profissional, período e estados corretos	3
US010	Cliente cancelar dentro da regra para liberar o horário	Cancelamento muda estado e cria uma oferta para a fila elegível	5
US011	Cliente entrar na espera para tentar um horário desejado	Serviço e janela ficam associados ao cliente e à ordem de entrada	3
US012	Sistema oferecer a vaga para preencher cancelamento	Próximo cliente compatível recebe oferta única com prazo	5
US013	Cliente aceitar ou recusar oferta para definir a reserva	Aceite dentro do prazo reserva atomicamente; recusa avança a fila	5
US014	Profissional encerrar atendimento para manter histórico	Concluído ou falta fica registrado sem apagar eventos anteriores	5
US015	Proprietária consultar ocupação para acompanhar operação	Período válido retorna totais reproduzíveis	3

RF016, RF017, RF019 e RF020 têm identificadores US016, US017, US019 e US020 no backlog, sem estimativa ou sprint. RF018 está fora do escopo. Apenas US001–US015 formam as 15 histórias estimadas.

Bloco 3 Arquitetura de software

ADR 001 Monólito modular. Uma aplicação Spring Boot organiza identidade, catálogo, agenda, agendamentos, lista de espera e relatórios. A equipe e o volume inicial não justificam microserviços. Transações locais mantêm reserva e oferta consistentes; módulos não acessam diretamente repositórios uns dos outros.

ADR 002 PostgreSQL 16. O domínio depende de relações, filtros de período e restrições contra sobreposição. PostgreSQL com `btree_gist`, intervalos `tstzrange` e Flyway protege a agenda. A alternativa NoSQL foi descartada porque consultas relacionais e integridade de horários são centrais.

ADR 003 REST JSON. Interface e API usam HTTPS, JSON e contrato OpenAPI 3.0 em `/api/v1`. Erros usam Problem Details e conflito de agenda retorna 409.

ADR 004 Sessões seguras. Cookie `HttpOnly`, `Secure` e `SameSite=Lax`, proteção CSRF para escrita, expiração por inatividade em 30 minutos e limite de 12 horas. O cliente autenticado consulta apenas seus próprios dados; papéis internos têm permissões distintas.

C4 contexto. Cliente consulta e administra a própria agenda; recepção configura serviços e marcações; profissional consulta e encerra atendimentos; proprietária acompanha operação. Todos usam Fila Certa. O provedor de e-mail não integra a V1.

C4 contêineres. Navegador web → interface HTML/CSS/JavaScript → API Spring Boot → PostgreSQL 16. A aplicação produz ofertas internas para a lista de espera; arquivos externos e mensageria não são necessários à V1.

C4 componentes. Identity aplica sessão e RBAC; Catalog mantém serviços e profissionais; Availability calcula janelas livres; Booking cria e cancela agendamentos; Waitlist escolhe e controla ofertas; Reporting agrega dados. Booking e Waitlist usam os serviços de disponibilidade e persistência por transação.

C4 código. `BookingController.create()` chama `BookingService.create()`, que valida cliente, serviço, profissional, expediente e bloqueios; o repositório tenta gravar o intervalo protegido por exclusão GiST. Em conflito, nenhuma reserva parcial é criada e a API devolve 409. `WaitlistService.offerNext()` usa bloqueio transacional para impedir ofertas simultâneas à mesma vaga.

Bloco 4 Diagramas UML

Os diagramas correspondentes aparecem nas páginas seguintes do Word. Eles usam os mesmos papéis, nomes de estados e decisões do modelo de dados e da API.

Casos de uso. ADMIN administra contas; proprietária e recepção organizam catálogo e agenda; profissional consulta e encerra seu atendimento; cliente consulta disponibilidade, marca, cancela e responde à oferta. `Cancelar agendamento` aciona a busca do próximo elegível na lista; `Aceitar oferta` confirma o horário enquanto o prazo estiver válido.

Classes. `Appointment` liga cliente, serviço e profissional com início, fim e estado; `AvailabilityRule` define expediente semanal; `ScheduleException` representa folga ou bloqueio; `WaitlistEntry` registra preferência e ordem; `WaitlistOffer` controla a vaga e seu vencimento.

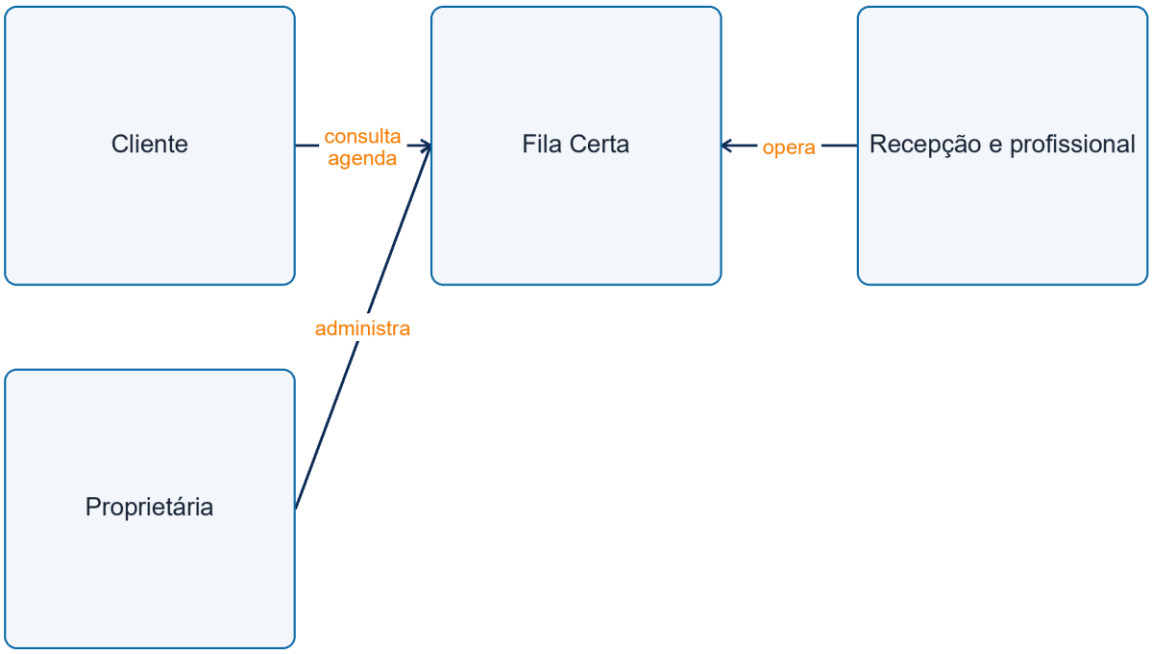
Componentes e deployment. A interface consome a API; API e banco são privados entre si, e só HTTPS fica exposto. DEV usa UI localhost:3000, API 8080 e PostgreSQL 5432. STAGING/PROD usam uma origem HTTPS atrás de reverse proxy; a instância simples é risco aceito para piloto, não SLA.

Sequência principal. Cliente → BookingController → BookingService → AvailabilityService → AppointmentRepository → PostgreSQL. A transação valida a disponibilidade e cria o agendamento; a restrição de intervalo decide conflitos simultâneos. Resposta: 201 ou 409, sem marcação duplicada.

Sequência de cancelamento e lista de espera. Cliente cancela → sistema marca CANCELLED e libera o intervalo → WaitlistService seleciona a entrada WAITING mais antiga compatível → cria oferta OFFERED com validade de duas horas → cliente aceita ou recusa no painel. Aceite reserva na mesma transação; expiração ou recusa marca a oferta e tenta o próximo. Sem pessoa elegível, o horário permanece livre.

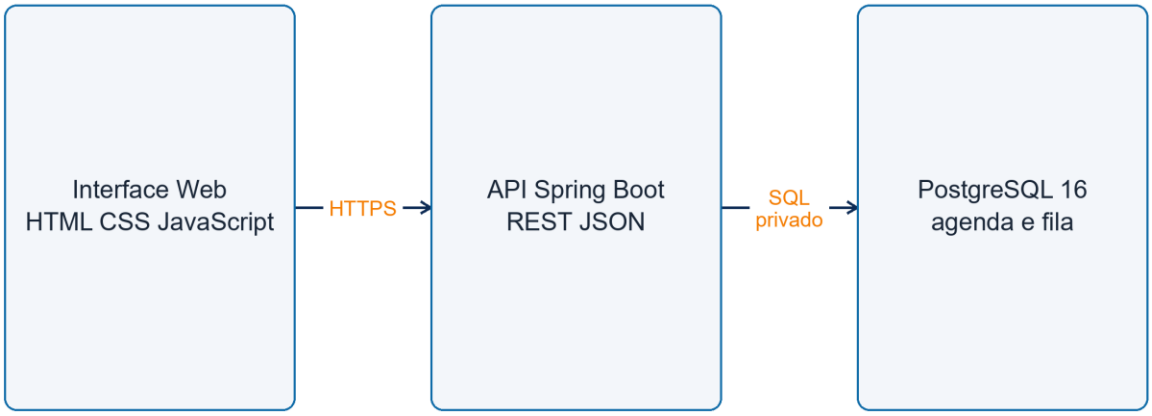
Atividades. Configurar serviço e disponibilidade → consultar horários → marcar; e cancelar → localizar fila compatível → oferecer → aceitar/recusar/expirar. O fluxo administrativo mostra ocupação ou estado vazio.

C4 Nível 1 Contexto



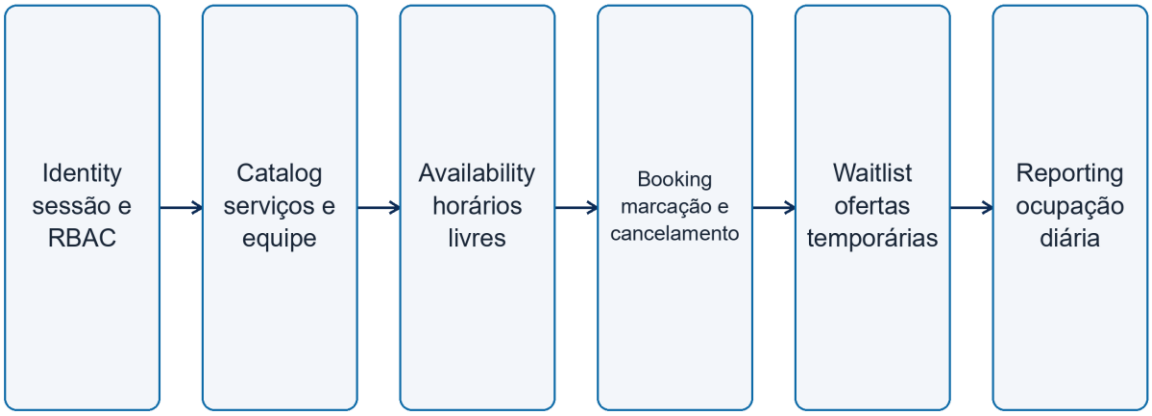
Tatyanne Guedes • Lavynia Monteiro • Daniel Costa | 09/11/2026

C4 Nível 2 Contêineres



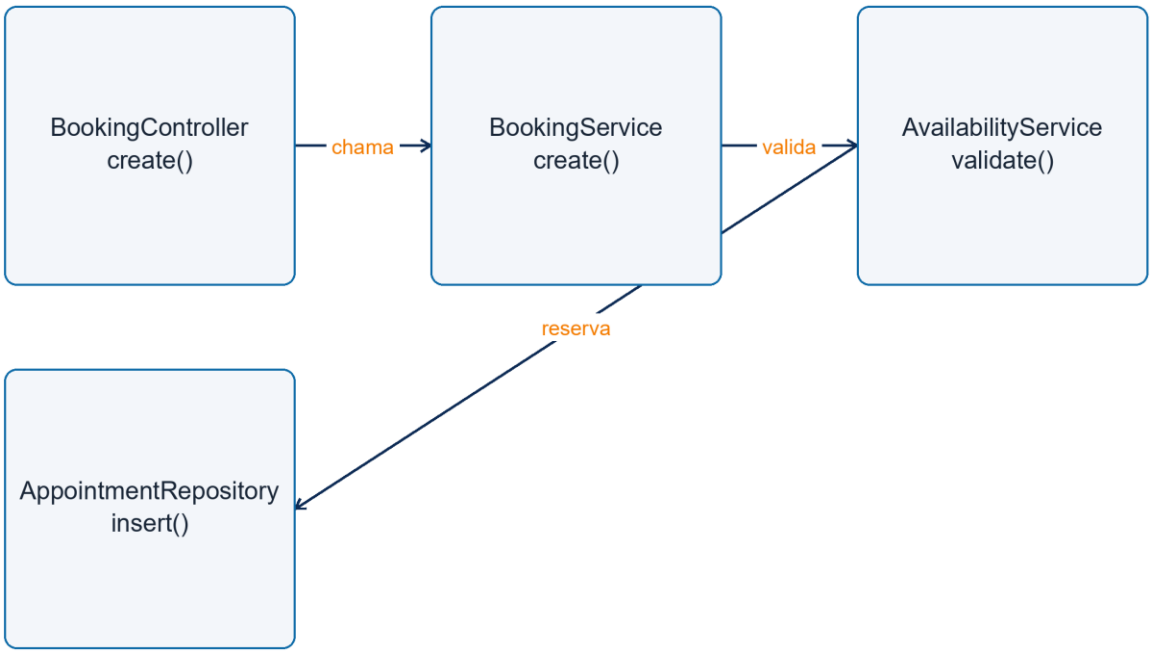
Tatyanne Guedes • Lavynia Monteiro • Daniel Costa | 09/11/2026

C4 Nível 3 Componentes



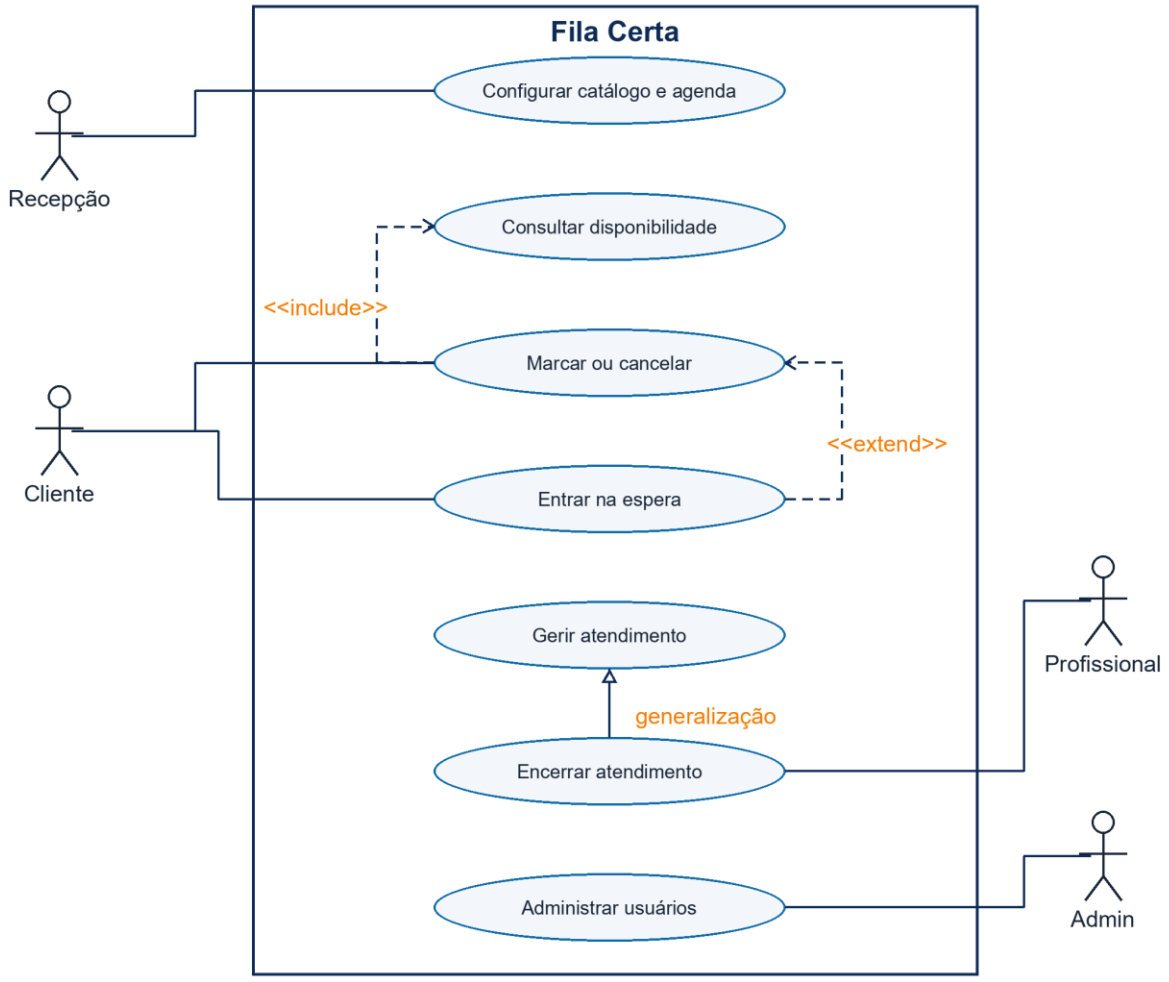
Tatyanne Guedes • Lavynia Monteiro • Daniel Costa | 09/11/2026

C4 Nível 4 Código BookingService



Tatyanne Guedes • Lavynia Monteiro • Daniel Costa | 09/11/2026

UML Casos de Uso — include, extend e generalização



Tracejado: <<include>> e <<extend>> | Triângulo vazio: generalização

Tatyanne Guedes • Lavynia Monteiro • Daniel Costa | 09/11/2026

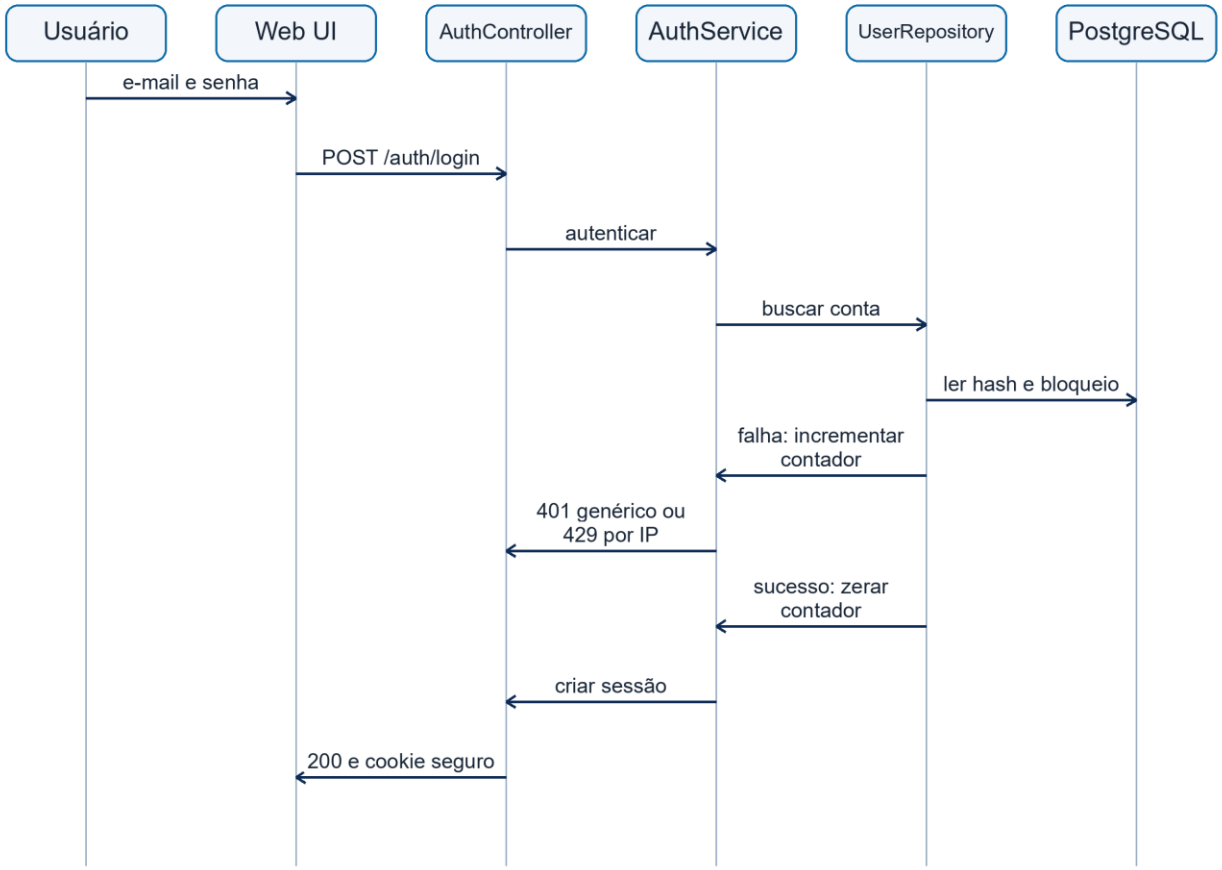
UML Classes — Agenda e Lista de Espera



UML: + público, - privado; atributos tipados e métodos com retorno.
Profissional N:N Serviço via ProfessionalService; AuditEvent é polimórfico por (entity_type, entity_id).

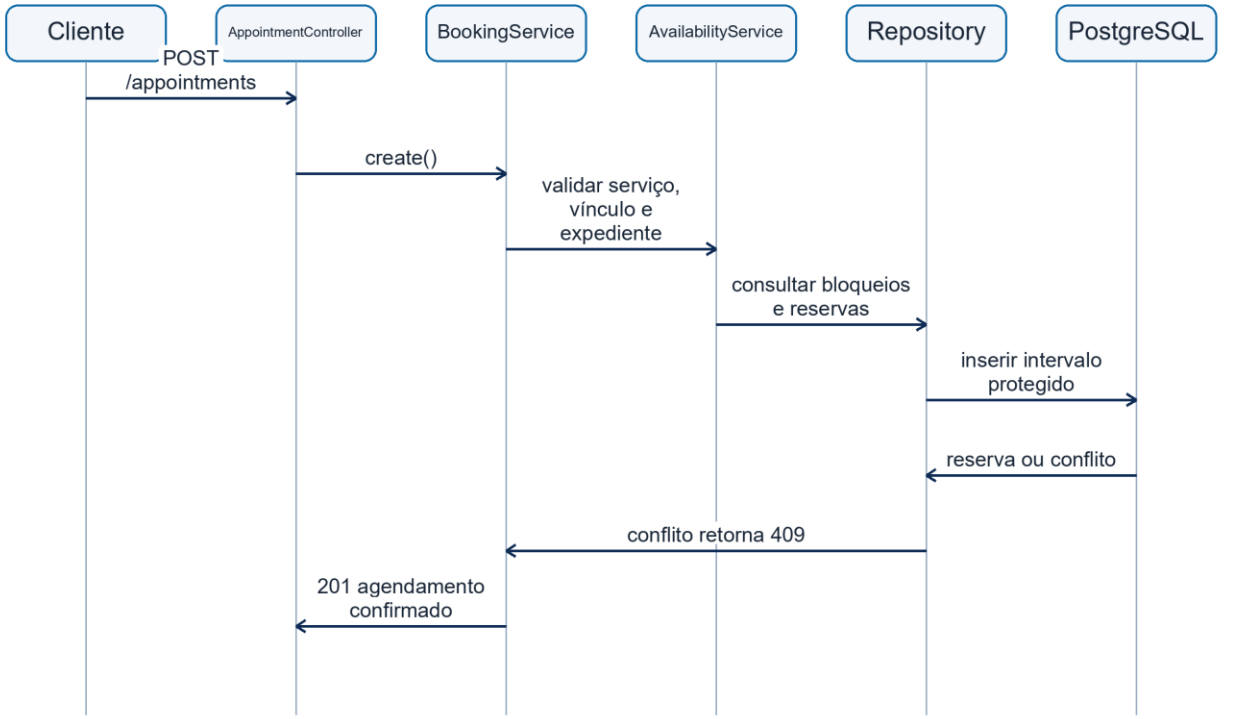
Tatyanne Guedes • Lavynia Monteiro • Daniel Costa | 09/11/2026

UML Sequência Autenticação



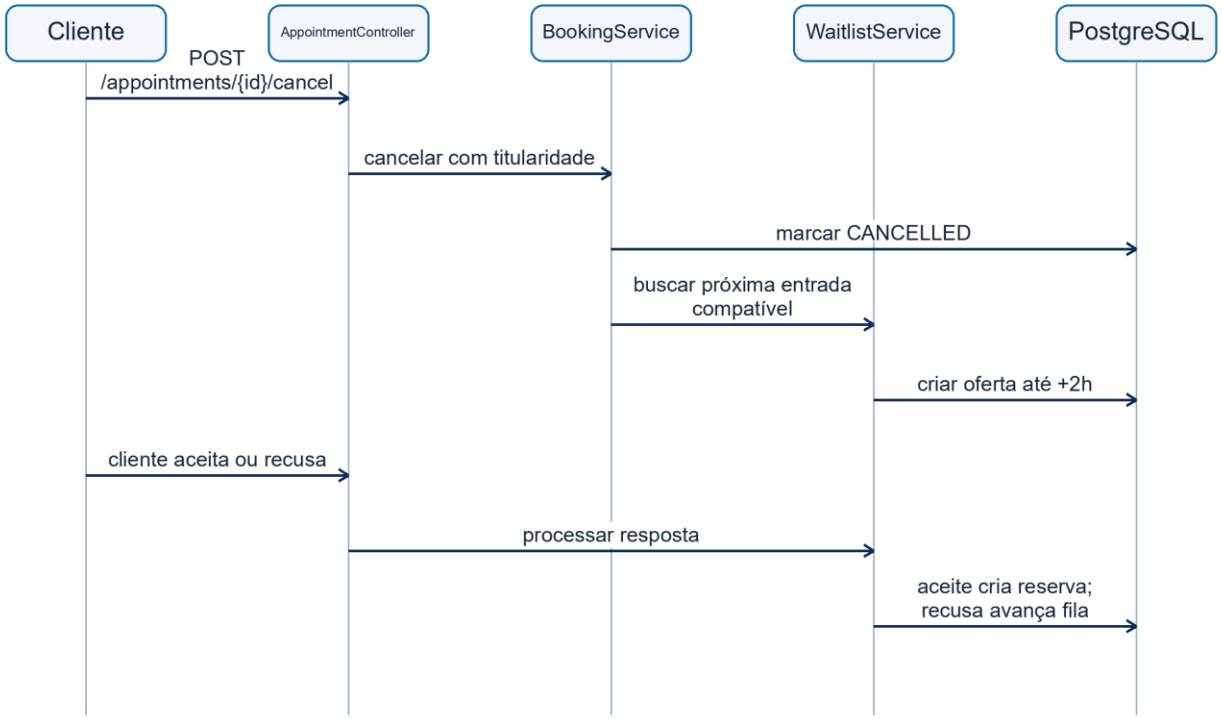
Tatyanne Guedes • Lavynia Monteiro • Daniel Costa | 09/11/2026

UML Sequência Marcação



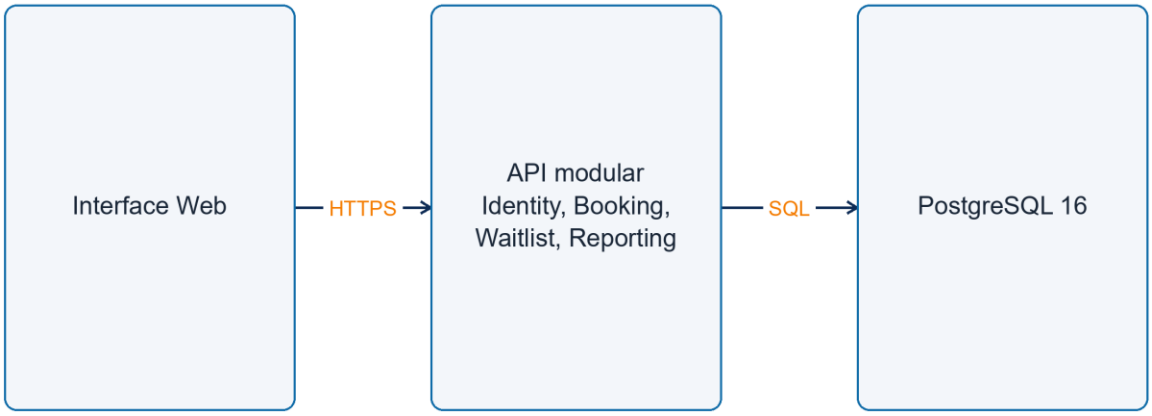
Tatyanne Guedes • Lavynia Monteiro • Daniel Costa | 09/11/2026

UML Sequência Cancelamento e Lista



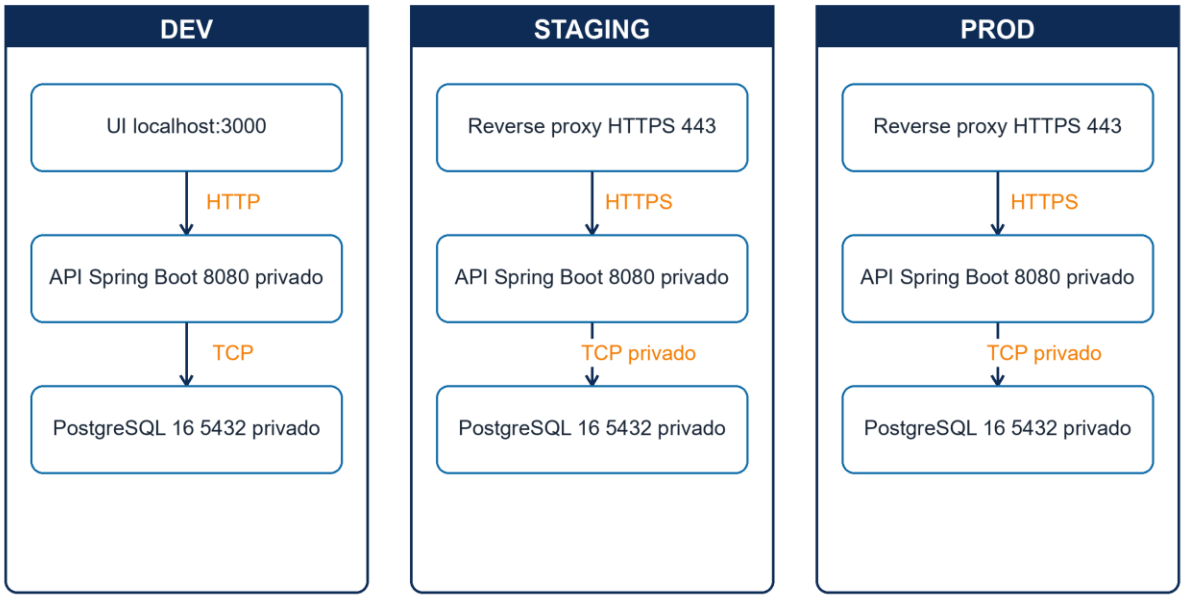
Tatyanne Guedes • Lavynia Monteiro • Daniel Costa | 09/11/2026

UML Componentes



Tatyanne Guedes • Lavynia Monteiro • Daniel Costa | 09/11/2026

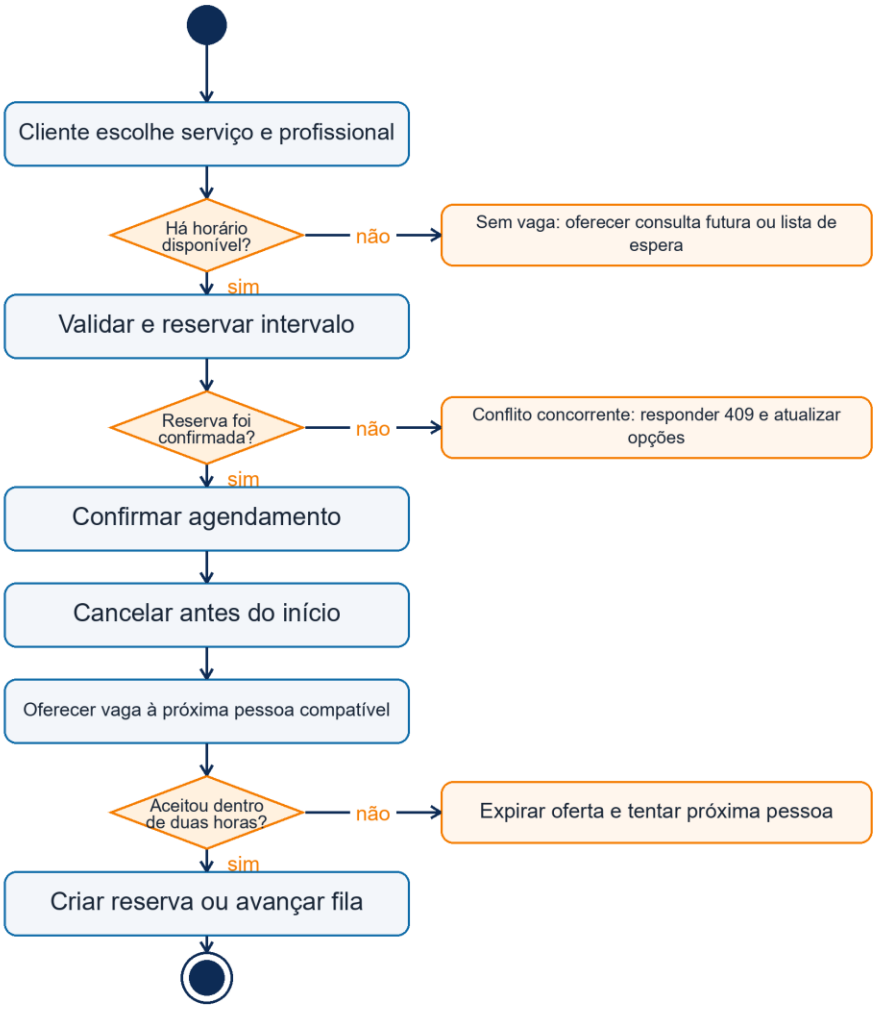
UML Deployment DEV, STAGING e PROD



PROD: banco privado, TLS obrigatório e backup diário; a instância simples é risco aceito para o piloto, sem SLA.
Pipeline promove a mesma imagem entre DEV, STAGING e PROD; configuração e credenciais são específicas por ambiente.

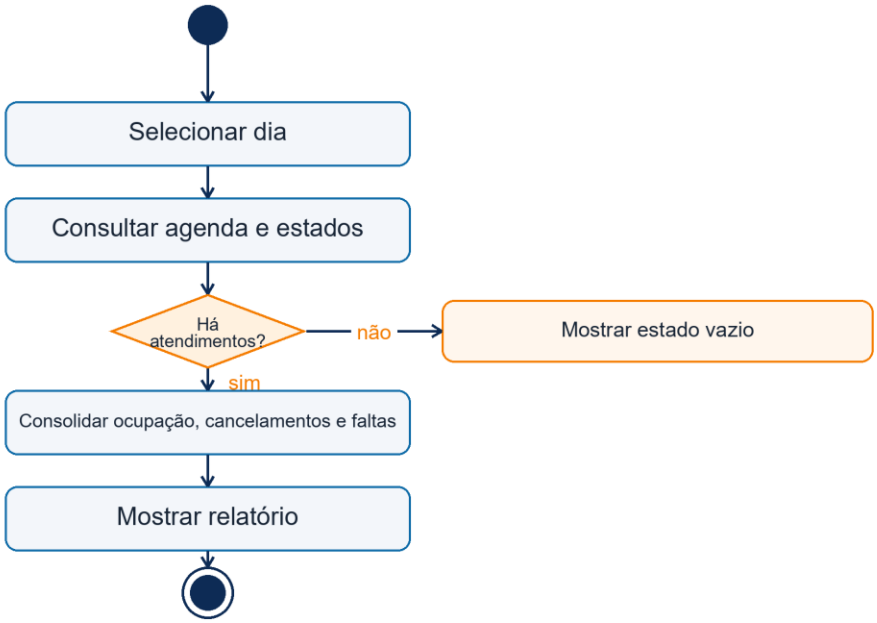
Tatyanne Guedes • Lavynia Monteiro • Daniel Costa | 09/11/2026

UML Atividade Agendamento e Lista de Espera



Tatyanne Guedes • Lavynia Monteiro • Daniel Costa | 09/11/2026

UML Atividade Administrativa



Tatyanne Guedes • Lavynia Monteiro • Daniel Costa | 09/11/2026

Dicionário de dados completo

Tabela	Campo	Tipo e restrições	Uso de negócio	Classificação
app_user	id	uuid PRIMARY KEY DEFAULT gen_random_uuid()	Identificador interno da linha.	Operacional
app_user	email	text NOT NULL UNIQUE	Contato eletrônico ou login.	PII
app_user	password_hash	text NOT NULL	Hash da senha; nunca é retornado pela API.	Credencial
app_user	role	text NOT NULL CHECK (role IN ('ADMIN','RECEPTIONIST','PROFESSIONAL','CUSTOMER'))	Papel de autorização do usuário.	Operacional
app_user	active	boolean NOT NULL DEFAULT true	Indica disponibilidade cadastral.	Operacional
app_user	failed_login_attempts	smallint NOT NULL DEFAULT 0 CHECK (failed_login_attempts BETWEEN 0 AND 5)	Falhas consecutivas; ao atingir cinco, iniciar bloqueio temporário.	Operacional
app_user	locked_until	timestamptz	Instante até o qual a conta não aceita login.	Operacional

Tabela	Campo	Tipo e restrições	Uso de negócio	Classificação
app_user	created_at	timestamptz NOT NULL DEFAULT now()	Data e hora da criação do registro.	Operacional
app_user	updated_at	timestamptz NOT NULL DEFAULT now()	Data e hora da última atualização.	Operacional
customer	id	uuid PRIMARY KEY DEFAULT gen_random_uuid()	Identificador interno da linha.	Operacional
customer	user_id	uuid NOT NULL UNIQUE REFERENCES app_user(id)	Conta de acesso ligada ao perfil.	Operacional
customer	full_name	text NOT NULL	Nome civil ou nome de atendimento da pessoa.	PII
customer	phone	text	Telefone de contato opcional do cliente.	PII
customer	created_at	timestamptz NOT NULL DEFAULT now()	Data e hora da criação do registro.	Operacional
professional	id	uuid PRIMARY KEY DEFAULT gen_random_uuid()	Identificador interno da linha.	Operacional
professional	user_id	uuid NOT NULL UNIQUE REFERENCES app_user(id)	Conta de acesso ligada ao perfil.	Operacional
professional	display_name	text NOT NULL	Nome profissional exibido na agenda.	PII
professional	active	boolean NOT NULL DEFAULT true	Indica disponibilidade cadastral.	Operacional
professional	created_at	timestamptz NOT NULL DEFAULT now()	Data e hora da criação do registro.	Operacional
service	id	uuid PRIMARY KEY DEFAULT gen_random_uuid()	Identificador interno da linha.	Operacional
service	name	text NOT NULL UNIQUE	Nome exibido na operação.	Operacional
service	description	text	Campo operacional da entidade.	Operacional
service	duration_minutes	integer NOT NULL CHECK (duration_minutes BETWEEN 5 AND 480)	Duração normal do serviço em minutos.	Operacional
service	preparation_minutes	integer NOT NULL DEFAULT 0 CHECK (preparation_minutes BETWEEN 0 AND 120)	Intervalo de preparação após o atendimento.	Operacional
service	price	numeric(10,2) CHECK (price IS NULL OR price >= 0)	Preço informativo do serviço, sem cobrança pela V1.	Operacional
service	active	boolean NOT NULL DEFAULT true	Indica disponibilidade cadastral.	Operacional
service	created_at	timestamptz NOT NULL DEFAULT now()	Data e hora da criação	Operacional

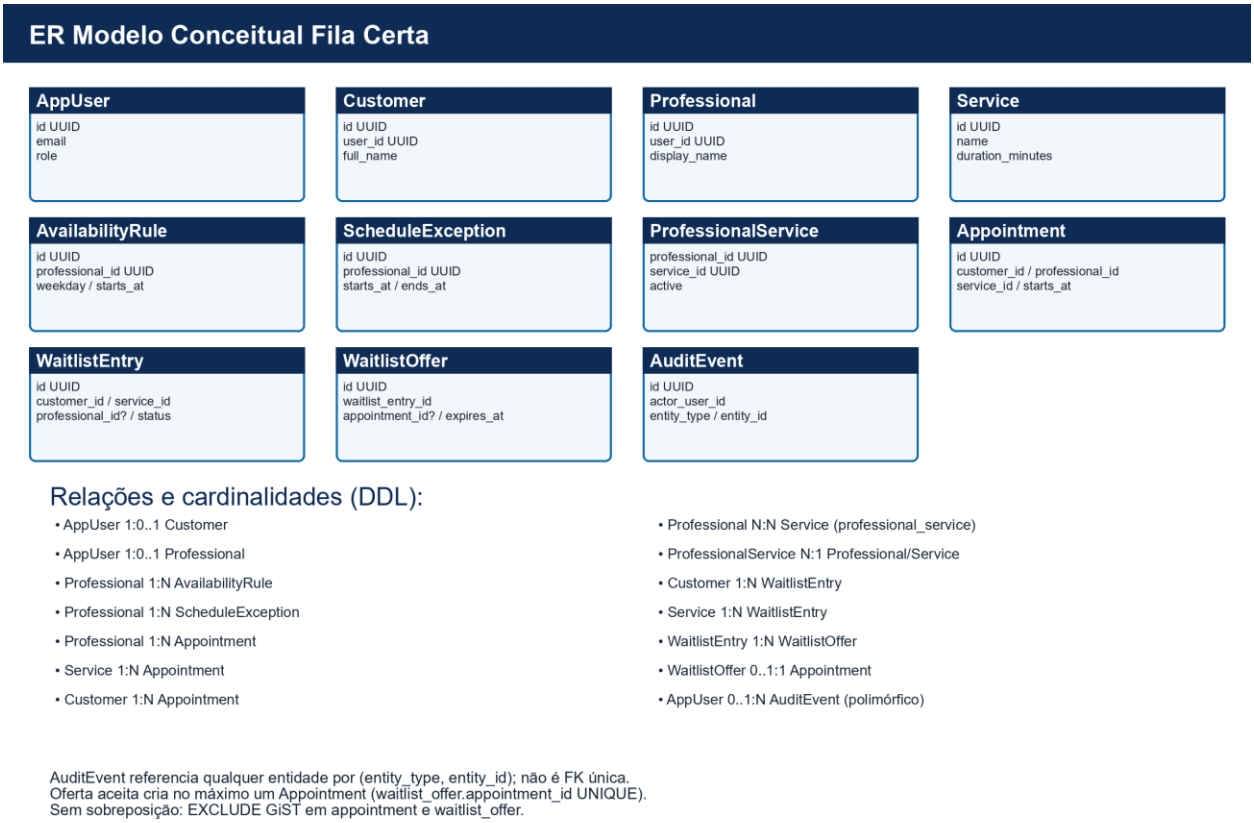
Tabela	Campo	Tipo e restrições	Uso de negócio	Classificação
			do registro.	
professional_service	professional_id	uuid NOT NULL REFERENCES professional(id)	Profissional responsável ou associado.	Operacional
professional_service	service_id	uuid NOT NULL REFERENCES service(id)	Serviço escolhido ou oferecido.	Operacional
professional_service	active	boolean NOT NULL DEFAULT true	Indica disponibilidade cadastral.	Operacional
availability_rule	id	uuid PRIMARY KEY DEFAULT gen_random_uuid()	Identificador interno da linha.	Operacional
availability_rule	professional_id	uuid NOT NULL REFERENCES professional(id)	Profissional responsável ou associado.	Operacional
availability_rule	weekday	smallint NOT NULL CHECK (weekday BETWEEN 0 AND 6)	Dia da semana conforme a convenção da aplicação.	Operacional
availability_rule	starts_at	time NOT NULL	Início do intervalo registrado.	Operacional
availability_rule	ends_at	time NOT NULL	Fim do intervalo registrado.	Operacional
availability_rule	valid_from	date NOT NULL	Início da validade da regra semanal.	Operacional
availability_rule	valid_until	date	Fim opcional da validade da regra semanal.	Operacional
schedule_exception	id	uuid PRIMARY KEY DEFAULT gen_random_uuid()	Identificador interno da linha.	Operacional
schedule_exception	professional_id	uuid NOT NULL REFERENCES professional(id)	Profissional responsável ou associado.	Operacional
schedule_exception	starts_at	timestamptz NOT NULL	Início do intervalo registrado.	Operacional
schedule_exception	ends_at	timestamptz NOT NULL	Fim do intervalo registrado.	Operacional
schedule_exception	kind	text NOT NULL DEFAULT 'BLOCK' CHECK (kind = 'BLOCK')	Tipo da exceção de agenda.	Operacional
schedule_exception	reason	text	Justificativa do movimento.	Operacional
appointment	id	uuid PRIMARY KEY DEFAULT gen_random_uuid()	Identificador interno da linha.	Operacional
appointment	customer_id	uuid NOT NULL REFERENCES customer(id)	Cliente dono da marcação ou entrada na lista.	Operacional
appointment	professional_id	uuid NOT NULL REFERENCES professional(id)	Profissional responsável ou associado.	Operacional
appointment	service_id	uuid NOT NULL REFERENCES service(id)	Serviço escolhido ou	Operacional

Tabela	Campo	Tipo e restrições	Uso de negócio	Classificação
			oferecido.	
appointment	starts_at	timestamptz NOT NULL	Início do intervalo registrado.	Operacional
appointment	ends_at	timestamptz NOT NULL	Fim do intervalo registrado.	Operacional
appointment	duration_minutes_snapshot	integer NOT NULL CHECK (duration_minutes_snapshot > 0)	Duração preservada no momento da marcação.	Operacional
appointment	preparation_minutes_snapshot	integer NOT NULL DEFAULT 0 CHECK (preparation_minutes_snapshot >= 0)	Intervalo preservado no momento da marcação.	Operacional
appointment	status	text NOT NULL DEFAULT 'BOOKED' CHECK (status IN ('BOOKED','CONFIRMED','CANCELLED','COMPLETED','NO_SHOW'))	Estado atual da entidade.	Operacional
appointment	cancellation_reason	text	Campo operacional da entidade.	Operacional
appointment	created_by	uuid NOT NULL REFERENCES app_user(id)	Campo operacional da entidade.	Operacional
appointment	created_at	timestamptz NOT NULL DEFAULT now()	Data e hora da criação do registro.	Operacional
appointment	updated_at	timestamptz NOT NULL DEFAULT now()	Data e hora da última atualização.	Operacional
waitlist_entry	id	uuid PRIMARY KEY DEFAULT gen_random_uuid()	Identificador interno da linha.	Operacional
waitlist_entry	customer_id	uuid NOT NULL REFERENCES customer(id)	Cliente dono da marcação ou entrada na lista.	Operacional
waitlist_entry	service_id	uuid NOT NULL REFERENCES service(id)	Serviço escolhido ou oferecido.	Operacional
waitlist_entry	professional_id	uuid REFERENCES professional(id)	Profissional responsável ou associado.	Operacional
waitlist_entry	desired_from	timestamptz NOT NULL	Início da janela desejada pela pessoa na espera.	Operacional
waitlist_entry	desired_until	timestamptz NOT NULL	Fim da janela desejada pela pessoa na espera.	Operacional
waitlist_entry	status	text NOT NULL DEFAULT 'WAITING' CHECK (status IN ('WAITING','OFFERED','FULFILLED','CANCELLED','EXPIRED'))	Estado atual da entidade.	Operacional
waitlist_entry	created_at	timestamptz NOT NULL DEFAULT now()	Data e hora da criação do registro.	Operacional
waitlist_offer	id	uuid PRIMARY KEY DEFAULT gen_random_uuid()	Identificador interno da linha.	Operacional
waitlist_offer	waitlist_entry_id	uuid NOT NULL REFERENCES waitlist_entry(id)	Entrada na fila atendida por esta oferta.	Operacional
waitlist_offer	appointment_id	uuid UNIQUE REFERENCES appointment(id)	Agendamento criado	Operacional

Tabela	Campo	Tipo e restrições	Uso de negócio	Classificação
			após aceite de uma oferta.	
waitlist_offer	professional_id	uuid NOT NULL REFERENCES professional(id)	Profissional responsável ou associado.	Operacional
waitlist_offer	starts_at	timestamptz NOT NULL	Início do intervalo registrado.	Operacional
waitlist_offer	ends_at	timestamptz NOT NULL	Fim do intervalo registrado.	Operacional
waitlist_offer	expires_at	timestamptz NOT NULL	Prazo para aceitar a oferta temporária.	Operacional
waitlist_offer	status	text NOT NULL DEFAULT 'OFFERED' CHECK (status IN ('OFFERED','ACCEPTED','DECLINED','EXPIRED'))	Estado atual da entidade.	Operacional
waitlist_offer	responded_at	timestamptz	Data e hora da resposta à oferta.	Operacional
waitlist_offer	created_at	timestamptz NOT NULL DEFAULT now()	Data e hora da criação do registro.	Operacional
audit_event	id	uuid PRIMARY KEY DEFAULT gen_random_uuid()	Identificador interno da linha.	Operacional
audit_event	actor_user_id	uuid REFERENCES app_user(id)	Usuário responsável pela ação.	PII
audit_event	entity_type	text NOT NULL	Tipo da entidade auditada.	Operacional
audit_event	entity_id	uuid NOT NULL	Identificador da entidade auditada.	Operacional
audit_event	action	text NOT NULL	Ação auditada.	Operacional
audit_event	occurred_at	timestamptz NOT NULL DEFAULT now()	Data e hora do fato ou evento.	Operacional
audit_event	details	jsonb NOT NULL DEFAULT '{}'::jsonb	Campo operacional da entidade.	Operacional

Bloco 5 Banco de dados

Modelo conceitual. Um cliente possui agendamentos e entradas na espera; um serviço pode ser executado por vários profissionais e vice-versa; um profissional tem regras semanais, exceções e agendamentos; cada oferta pertence a uma entrada da espera e a uma vaga liberada.



Tatyanne Guedes • Lavynia Monteiro • Daniel Costa | 09/11/2026

Entidade	Responsabilidade
`app_user`	credencial, papel e bloqueio de login
`customer`	dados mínimos do cliente ligado à conta
`professional`	perfil interno e estado ativo
`service`	nome, duração e intervalo de preparação
`professional_service`	associação habilitada entre profissional e serviço
`availability_rule`	expediente semanal local
`schedule_exception`	folga, bloqueio ou horário especial
`appointment`	reserva, estado e snapshot da duração
`waitlist_entry`	preferência do cliente, serviço, janela e posição temporal
`waitlist_offer`	oferta temporária de um horário e resposta do cliente
`audit_event`	alterações relevantes de conta e agenda

Leitura simples do modelo. Cada linha abaixo explica o papel dos dados principais sem exigir leitura de SQL. “Identificador” é o código único do registro; “referência” liga uma informação a outra tabela.

Tabela	Informações guardadas	Ligações importantes
Conta (`app_user`)	identificador, e-mail, senha protegida, papel e bloqueio de acesso	pode corresponder a um perfil de cliente ou profissional
Cliente (`customer`)	identificador da conta, nome e telefone	possui agendamentos e pode entrar na lista de espera
Profissional (`professional`)	identificador da conta, nome de exibição e situação ativa	atende serviços e tem expediente, exceções e agendamentos
Serviço (`service`)	nome, duração, intervalo de preparação e preço informativo	pode ser oferecido por vários profissionais
Expediente (`availability_rule`)	profissional, dia da semana, hora inicial e final	define quando aquele profissional pode atender normalmente
Exceção (`schedule_exception`)	profissional, período e tipo de bloqueio ou horário especial	altera o expediente normal em datas específicas
Agendamento (`appointment`)	cliente, profissional, serviço, início, fim e estado	ocupa um intervalo e mantém a duração acordada no momento da marcação
Entrada na espera (`waitlist_entry`)	cliente, serviço, período desejado e data de	organiza quem pode receber uma vaga que

Tabela	Informações guardadas	Ligações importantes
	entrada	abriu
Oferta (`waitlist_offer`)	entrada da espera, horário oferecido, prazo e resposta	se aceita, origina um agendamento
Auditoria (`audit_event`)	pessoa que fez a alteração, ação e momento	registra mudanças importantes para consulta administrativa

Regras que o banco protege. O banco impede dois agendamentos ativos do mesmo profissional no mesmo intervalo. Ao cancelar, o sistema libera o horário e registra a primeira oferta à fila como uma única operação. Uma oferta só pode ser aceita enquanto estiver no prazo e a vaga continuar livre. Se outra pessoa ocupar o horário primeiro, o sistema informa conflito e não cria uma segunda reserva.

Como ler o diagrama ER. As ligações “um para muitos” significam, por exemplo, que um cliente pode ter vários agendamentos, mas cada agendamento pertence a um cliente. “Muitos para muitos” significa que vários profissionais podem oferecer vários serviços; a associação entre eles é guardada em `professional_service`.

Integridade. `appointment` armazena início e fim como `timestamptz`; uma exclusão GiST impede sobreposição para o mesmo profissional quando o estado é BOOKED ou CONFIRMED. Duração e intervalo de preparação são copiados no agendamento para preservar o histórico se o catálogo mudar. Cancelamento e criação da primeira oferta ocorrem na mesma transação. Um único `waitlist_offer` ativo pode ocupar a vaga. Aceite verifica prazo, vaga ainda livre e elegibilidade sob lock; caso contrário retorna 409. Regras semanais e exceções são consideradas ao gerar disponibilidade.

Normalização e segurança. Dados de conta ficam separados do perfil de cliente e profissional. E-mail tem unicidade sem distinção de caixa. Senhas são armazenadas como hash; contador e horário de bloqueio sustentam RNF005. Não guardar dados de pagamento nem informações de saúde. Definir base legal, acesso e retenção antes do piloto real.

O diagrama ER e o dicionário de dados apresentam o modelo do banco neste documento, sem reproduzir os comandos de criação das tabelas. A estrutura foi planejada para PostgreSQL 16 e deve ser validada em ambiente de teste antes de qualquer uso compartilhado.

Bloco 7 Planejamento e estimativas

WBS. 1 Produto (visão, riscos, requisitos); 2 Arquitetura (decisões e diagramas); 3 Dados (modelo, dicionário e migration); 4 API e interface (contrato, telas e autenticação); 5 Agenda (disponibilidade, marcação, cancelamento); 6 Lista de espera (elegibilidade, oferta e aceite); 7 Qualidade (testes, documentação e homologação).

As 15 histórias planejadas somam 61 pontos relativos. A capacidade estimada é de 66 pontos no conjunto das três sprints (22 pontos por sprint de duas semanas), deixando uma margem de cinco pontos para ajustes. Pontos são estimativas relativas, não horas; a velocidade real só poderá ser confirmada durante o desenvolvimento.

Sprint	Histórias	SP	Resultado esperado
1 semanas 1–2	US001, US003–US007	22	acesso, cadastros e consulta de horários livres
2 semanas 3–4	US002, US008–US011	21	gestão de contas, marcação, agenda, cancelamento e fila
3 semanas 5–6	US012–US015	18	oferta e resposta da fila, encerramento e relatório

Matriz de rastreabilidade. “Planejado” não significa implementado. A sequência principal cobre consulta e marcação; cancelamento e oferta têm sequência própria.

RF	História/prioridade	API e diagrama	Dados e entrega
RF001	US001 Must S1	`POST /auth/login`; sequência login	`app_user`; S1
RF002	US002 Must S2	`/users`; casos de uso	`app_user`, `audit_event`; S2
RF003	US003 Must S1	`/customers`; classes	`customer`; S1
RF004	US004 Must S1	`/services`; classes	`service`; S1
RF005	US005 Must S1	`/professionals`, `/professional-services`; componentes	`professional`, `professional_service`; S1
RF006	US006 Must S1	`/availability-rules`, `/schedule-exceptions`; atividade	`availability_rule`, `schedule_exception`; S1
RF007	US007 Must S1	`GET /availability`; sequência principal	consulta regras, exceções e reservas; S1
RF008	US008 Must S2	`POST /appointments`; sequência principal	`appointment`; S2
RF009	US009 Must S2	`GET /appointments`; agenda	`appointment`; S2
RF010	US010 Must S2	`POST /appointments/{id}/cancel`; sequência de cancelamento	`appointment`, cria oferta; S2
RF011	US011 Must S2	`POST /waitlist`; classes	`waitlist_entry`; S2
RF012	US012 Must S3	`POST /waitlist/offers`; sequência de cancelamento	`waitlist_offer`; S3
RF013	US013 Must S3	`POST /waitlist/offers/{id}/accept e /decline`	`waitlist_offer`, `appointment`; S3
RF014	US014 Must S3	`PATCH /appointments/{id}/outcome`; atividade	`appointment`, `audit_event`; S3
RF015	US015 Should S3	`GET /reports/daily`; atividade administrativa	agregação de agenda; S3, sujeita à capacidade
RF016	US016 Should, backlog	`/customers/me` proposto, sem compromisso V1	`customer`; sem sprint
RF017	US017 Could, backlog	sem endpoint ou provedor na V1	sem tabela; sem sprint
RF018	Won't V1	sem endpoint de cobrança	sem dados financeiros
RF019	US019 Could, backlog	sem endpoint de exportação na V1	sem tabela; sem sprint
RF020	US020 Could, backlog	sem endpoint de painel na V1	agregação apenas no relatório diário

Bloco 8 Documentação para o desenvolvimento

O README, configuração, Git Flow, visão funcional do contrato da API e modelo visual do banco são apresentados neste documento para que a entrega não dependa de enviar um repositório. Os comandos de ambiente são instruções futuras; não afirmam que a aplicação já existe.

README inicial

Projeto: Fila Certa. **Objetivo:** organizar horários de serviços, evitar reservas sobrepostas e preencher cancelamentos com uma lista de espera. **Estrutura futura:** `backend/`, `frontend/`, `database/` e `docs/`. O modelo do banco e a visão funcional da API estão representados neste documento; os arquivos de implementação são materiais de apoio da equipe, não fazem parte da apresentação.

Passo a passo para clonar e iniciar o projeto. O repositório entregue nesta etapa contém a documentação, o contrato visual da API, o esquema do banco e scripts de geração dos documentos. A aplicação ainda não foi implementada. Portanto, os passos 1 a 3 abaixo funcionam com o repositório atual; os passos 4 a 8 descrevem a sequência que deverá ser seguida quando as pastas de aplicação da Sprint 1 forem criadas.

1. **Clonar a documentação.** Instale Git e Python 3.12; no terminal, execute `git clone https://github.com/lavyniamonteiro/LogiGoiana.git`, depois `cd LogiGoiana`.

2. **Preparar ambiente local.** Copie `.env.example` para `.env` (`Copy-Item .env.example .env` no PowerShell ou `cp .env.example .env` no macOS/Linux). Substitua senhas demonstrativas por valores locais e não envie `.env` ao Git. No estado atual, esse arquivo serve de referência; a aplicação ainda não o lê.

3. **Conferir artefatos existentes.** Valide o contrato com `python scripts/validate_openapi.py`. Depois que PostgreSQL 16 estiver instalado e um banco de teste vazio tiver sido criado, aplique o modelo com `psql -v ON_ERROR_STOP=1 -d fila_certa -f database/V1__core.sql`. Esses comandos validam somente a especificação e o esquema; não iniciam uma aplicação.

4. **Subir o banco quando a configuração Compose da Sprint 1 estiver disponível.** Execute `docker compose up -d db` e confirme que o serviço está saudável com `docker compose ps`. O arquivo Compose ainda precisa ser criado na implementação.

5. **Preparar a API quando o projeto Maven existir.** Na pasta do backend, copie a configuração local para `.env`, confira a conexão com o banco e aplique as migrations. Inicie com `./mvnw spring-boot:run` (Windows: `mvnw.cmd spring-boot:run`). A API planejada ficará na porta 8080.

6. **Subir a interface quando os arquivos estáticos existirem.** Na pasta do frontend, execute `python -m http.server 3000` para servir a interface planejada em HTML, CSS e JavaScript. A tela ficará disponível em `http://localhost:3000` e usará a API na porta 8080.

7. **Executar verificações.** No backend, rode `./mvnw test` (Windows: `mvnw.cmd test`). Depois, abra a interface, entre com uma conta de teste e percorra consulta de horários, marcação e cancelamento. As pastas e os comandos de aplicação dos passos 4 a 7 só funcionarão depois de implementados; não estão presentes nesta entrega documental.

8. **Encerrar os serviços locais.** Pare API e servidor web com `Ctrl+C`; quando Compose existir, encerre os contêineres com `docker compose down`.

Nunca use dados pessoais reais no ambiente local. A conta ADMIN inicial deve ser criada por procedimento seguro de desenvolvimento, ainda a definir na implementação.

Configuração de referência

Variável	Exemplo	Uso
APP_ENV	local	perfil de execução
DB_URL	jdbc:postgresql://localhost:5432/filacerta	conexão local
DB_USER	filacerta_app	usuário do banco
DB_PASSWORD	replace-locally	segredo local ilustrativo
SESSION_SECRET	replace-with-random-secret	chave de sessão
CORS_ALLOWED_ORIGINS	http://localhost:3000	UI local autorizada
APP_TIME_ZONE	America/Sao_Paulo	fuso do estabelecimento piloto

Git Flow

`main` contém versões revisadas. Criar branch curta, como `feat/RF008-agendamento`, `fix/RF010-cancelamento` ou `docs/diagrama-espera`. Fazer commits pequenos (`feat: criar agendamento`, `fix: impedir sobreposicao`). Revisar

mudança, executar testes e integrar por merge. Tags identificam entregas como `v0.1.0`. O remoto GitHub pode ser usado pela equipe, mas não faz parte dos artefatos que precisam ser enviados ao professor.

Contrato resumido da API

O que é. A API é a porta de comunicação entre a tela e as regras do sistema. O contrato informa que ações podem ser solicitadas, quais dados são necessários e o que a tela recebe em resposta. A tabela abaixo mostra esse acordo em linguagem funcional, sem listar rotas ou trechos de especificação.

Ação solicitada pela tela	Quem pode solicitar	Informações necessárias	O que o sistema devolve
Criar conta ou entrar	Cliente	nome, e-mail e senha; no acesso, e-mail e senha	Confirmação de cadastro ou acesso e aviso se houver erro
Consultar horários livres	Cliente ou recepção	serviço, período desejado e profissional opcional	Lista de horários compatíveis
Marcar horário	Cliente ou recepção	cliente, serviço, profissional e horário escolhido	Confirmação da reserva ou aviso de conflito
Cancelar horário	Cliente responsável ou recepção	identificação do agendamento	Confirmação do cancelamento e início da busca na fila
Entrar na lista de espera	Cliente ou recepção	serviço e período de preferência	Confirmação de entrada na fila
Responder a uma oferta	Cliente destinatário	escolha de aceitar ou recusar	Reserva confirmada, ou tentativa de oferecer a vaga à próxima pessoa
Administrar agenda e equipe	Recepção ou administração	serviços, profissionais, expediente e exceções	Confirmação da alteração ou aviso para corrigir os dados
Encerrar atendimento e consultar resumo diário	Profissional ou recepção	resultado do atendimento ou dia consultado	Registro atualizado ou resumo da agenda

Fluxo visual da consulta e marcação: Pessoa escolhe serviço e período → Sistema verifica expediente, folgas e reservas → Tela apresenta os horários livres → Pessoa escolhe um horário → Sistema confere novamente e confirma ou explica que a vaga acabou de ser ocupada.

Respostas e avisos. Uma resposta de sucesso confirma a ação e devolve as informações para atualizar a tela. Se algo não puder ser feito, o sistema explica o motivo: dados incompletos, acesso não autorizado, item não encontrado, horário ocupado, regra não atendida ou muitas tentativas de acesso. A pessoa recebe uma orientação clara sobre o próximo passo.

Guia de padrões de código

Área	Padrão combinado	Exemplo de aplicação
Nomes e organização	Classes em PascalCase; métodos e atributos em camelCase; nomes em inglês para coincidir com o modelo e a API. Separar pacotes por domínio: identidade, catálogo, disponibilidade, agendamento e espera.	`AppointmentService` concentra regras de agendamento.
Camadas	Controllers recebem solicitações e validam formato; services aplicam regras; repositories consultam e gravam dados. Controller não chama repository diretamente.	A tela solicita o agendamento ao controller, que delega a validação ao service.
Entrada e saída	Usar objetos de requisição e resposta próprios; validar campos obrigatórios, formatos e limites antes de executar regras. Não devolver senha, hash ou campos internos.	Resposta de cliente contém identificador, nome e contato autorizado.
Datas e horários	Instantes de agendamento guardam fuso; expediente semanal usa hora local do estabelecimento. Converter para o fuso configurado ao exibir.	Um horário marcado continua correto mesmo após conversão entre fusos.
Persistência	Mudanças de estrutura são migrations numeradas e revisadas; migrations já compartilhadas não são reescritas. Usar transação para marcar, cancelar e aceitar oferta.	A confirmação e a reserva acontecem juntas ou nenhuma é gravada.
Erros e segurança	Responder com erro consistente, sem expor detalhes internos; conferir papel e titularidade no servidor em toda operação privada.	Cliente só cancela o próprio horário.
Testes e revisão	Nomear testes pelo comportamento	Duas solicitações simultâneas não podem

Área	Padrão combinado	Exemplo de aplicação
	esperado; cobrir sucesso, erro, permissão e concorrência. Formatar antes de abrir revisão e não mesclar sem aprovação da equipe.	ocupar o mesmo profissional.

Esses padrões descrevem a convenção planejada; ainda não há código de aplicação para verificá-los.

Sprint 1 detalhada

Objetivo: estabelecer acesso seguro, cadastros básicos e consulta confiável de horários livres. A sprint inclui seis histórias, totalizando 22 pontos, igual à capacidade planejada para duas semanas. O cliente ainda não cria agendamentos nesta sprint; essa função entra na Sprint 2.

História	Trabalho incluído	Critérios de aceite	Estimativa
US001 Entrar com segurança	Criar acesso por e-mail e senha, manter sessão e aplicar papéis definidos.	Credenciais corretas abrem sessão com o papel correspondente; após cinco falhas, a conta fica bloqueada por 15 minutos; acesso inválido não revela se o e-mail existe.	3 SP
US003 Cadastrar cliente	Registrar o perfil mínimo necessário para uma marcação futura e ligá-lo à conta do cliente.	Campos obrigatórios são validados; e-mail não pode duplicar; perfil fica ligado a uma única conta e não expõe dados de outra pessoa.	3 SP
US004 Cadastrar serviço	Registrar nome, duração e intervalo de preparação.	Nome é obrigatório; duração deve ser positiva; preparação não pode ser negativa; serviço inativo deixa de aparecer em novas consultas sem apagar histórico.	3 SP
US005 Associar profissional e serviço	Indicar quais serviços cada profissional pode executar.	Só profissionais ativos podem ser associados; não há associação duplicada; consulta de disponibilidade respeita apenas associações habilitadas.	3 SP
US006 Configurar expediente e exceções	Definir horários semanais e registrar folgas, bloqueios ou horários especiais.	Início deve anteceder fim; exceção prevalece sobre expediente normal; períodos bloqueados não aparecem na consulta; alterações ficam registradas para auditoria.	5 SP
US007 Consultar horários livres	Gerar opções compatíveis com serviço, profissional e período.	Duração e preparação cabem no expediente; serviço está associado ao profissional; exceções e agendamentos existentes são respeitados; períodos passados não são oferecidos.	5 SP

Sequência de execução: criar base e migrations → implementar login e papéis → cadastrar cliente, serviços e profissionais → configurar expediente e exceções → calcular e apresentar horários disponíveis → revisar os critérios e demonstrar o fluxo em homologação.

Saída da sprint: equipe consegue entrar, configurar a oferta e consultar opções válidas. A demonstração usa dados fictícios. Os critérios ainda serão confirmados por testes após a aplicação existir.

Bloco 9 Qualidade e processo

Estratégia de testes. Os testes são verificações planejadas para encontrar erros antes da entrega. A tabela explica cada camada sem mostrar comandos ou código.

Tipo de verificação	O que confere no Fila Certa	Exemplo de resultado esperado
Teste de regra	Uma decisão isolada do sistema	Um horário que já passou não aparece como disponível
Teste entre partes	A comunicação entre sistema e banco	Duas tentativas no mesmo horário não criam reservas duplicadas
Teste da API	Permissões, dados recebidos e respostas	Cliente não consulta dados privados de outro cliente
Teste do fluxo completo	A jornada pela tela, do começo ao fim	A pessoa consulta, marca, cancela e recebe uma oferta da fila

Os testes devem cobrir ao menos 70% das regras de negócio e todas as situações críticas de marcação, cancelamento e aceite da oferta. Devem incluir horários simultâneos, tentativas sem permissão, expiração e recusa. São atividades planejadas; não afirmamos que foram executadas.

Ferramentas e comandos planejados. JUnit 5 e Mockito verificam regras e serviços; Spring Boot Test verifica a integração da API; Testcontainers inicia um PostgreSQL temporário para conferir transações e concorrência; Playwright percorre a jornada pela interface. Depois que o backend Java e o wrapper Maven forem criados, a equipe poderá executar `./mvnw test` (Windows: `mvnw.cmd test`). A suíte de interface será executada com `npx playwright test`. A cobertura das regras será conferida com JaCoCo e `./mvnw verify`. Esses comandos são o plano da equipe; ainda não podem ser executados porque os módulos da aplicação não existem nesta entrega documental.

Pronto para começar. Uma atividade pode entrar na sprint quando tem objetivo claro, pessoa usuária identificada, critérios de aceite compreendidos, dependências conhecidas e tamanho estimado pela equipe.

Pronto para entregar. Uma atividade pode ser considerada concluída quando atende aos critérios combinados, passou pelas verificações previstas, respeita as permissões, não apresenta defeito crítico conhecido e tem documentação e diagramas atualizados quando necessário. A equipe demonstra o resultado em um ambiente de homologação.

Processo escolhido. O planejamento usa Scrum com sprints de duas semanas porque o trabalho acadêmico tem entregas incrementais, prioridades e uma data de apresentação. Ao fim de cada sprint, a equipe revisa o que foi concluído e ajusta a próxima etapa. A velocidade é uma estimativa inicial, não uma promessa de produtividade.

Glossário do domínio. Disponibilidade é o intervalo em que um profissional pode atender. Exceção é uma folga ou bloqueio que altera o expediente normal. Agendamento é a reserva confirmada de um intervalo. Lista de espera organiza pessoas interessadas em um serviço e período. Oferta é uma oportunidade temporária criada quando uma vaga abre. Conflito é uma tentativa de ocupar um período já reservado.

Anexos visuais

O modelo do banco é apresentado pelo diagrama de entidades e relacionamentos, acompanhado pela explicação das tabelas e regras principais. O contrato da API é apresentado pela tabela de ações, dados necessários e respostas esperadas, além do fluxo visual de consulta e marcação. O documento não reproduz listagens de SQL ou OpenAPI; elas não são necessárias para explicar a proposta na apresentação.

Anexo visual do banco de dados

O diagrama mostra as principais informações do sistema e como elas se ligam. O dicionário de dados no Bloco 5 explica os campos em linguagem simples.



Relações e cardinalidades (DDL):

- AppUser 1:0..1 Customer
- AppUser 1:0..1 Professional
- Professional 1:N AvailabilityRule
- Professional 1:N ScheduleException
- Professional 1:N Appointment
- Service 1:N Appointment
- Customer 1:N Appointment
- Professional N:N Service (professional_service)
- ProfessionalService N:1 Professional/Service
- Customer 1:N WaitlistEntry
- Service 1:N WaitlistEntry
- WaitlistEntry 1:N WaitlistOffer
- WaitlistOffer 0..1:1 Appointment
- AppUser 0..1:N AuditEvent (polimórfico)

AuditEvent referencia qualquer entidade por (entity_type, entity_id); não é FK única.
Oferta aceita cria no máximo um Appointment (waitlist_offer.appointment_id UNIQUE).
Sem sobreposição: EXCLUDE GiST em appointment e waitlist_offer.

Tatyanne Guedes • Lavynia Monteiro • Daniel Costa | 09/11/2026

Anexo visual do contrato da API

A API recebe solicitações da tela, aplica as regras da agenda e devolve uma confirmação, opções ou uma orientação de correção.

Pessoa escolhe serviço e período	Sistema confere expediente e reservas	Tela mostra horários livres	Pessoa seleciona um horário	Sistema confirma ou avisa conflito

O sistema verifica a disponibilidade mais uma vez no momento da marcação. Assim, se duas pessoas escolherem o mesmo horário quase ao mesmo tempo, apenas uma reserva é confirmada; a outra recebe um aviso para escolher novamente.